MAPL3 Child Component Creation Flow - Complete Analysis

Overview

This document describes how child grid components are created from parent grid components in MAPL3 (develop branch). The process is driven by specifications in the **parent's YAML configuration file**, where the parent declares all child components along with their shared object (.so) files and SetServices routine names.

Current Architecture (Parent-Driven)

- **Responsibility:** Parent YAML defines all child details
- **Information Location:** Parent's YAML `children:` section
- **Discovery:** Parent reads its own YAML and creates children
- **Key Characteristic:** Parent knows about all its children upfront

---

Step-by-Step Flow

Step 1: Parent YAML Structure and Storage

File: `/Users/wdboggs/src/MAPL/superstructure/generic/GenericGridComp.F90` (Lines 87-142)

Parent's YAML is loaded as an `ESMF_HConfig` object in `create_grid_comp_primary()` and stored in `OuterMetaComponent`:

```
 recursive type(ESMF_GridComp) function create_grid_comp_primary( &
      name, set_services, config, unusable, petlist, rc) result(gridcomp)
   type(ESMF_HConfig), intent(in) :: config          ! <-- YAML loaded as HConfig
```

```
  ! Store HConfig in OuterMetaComponent
  outer_meta = OuterMetaComponent(gridcomp, user_gc_driver, set_services, config)
end function create_grid_comp_primary
```

Example Parent YAML Structure (`scenarios/precision_extension/parent.yaml`):

```yaml
grid:
  class: LatLon
  im_world: 12
  jm_world: 6

children:
  A:
    dso: libconfigurable_gridcomp
    config_file: scenarios/precision_extension/A.yaml
  B:
    dso: libconfigurable_gridcomp
    config_file: scenarios/precision_extension/B.yaml

states: {}

connections:
  - src_name: E_A1
    dst_name: I_B1
    src_comp: A
```

```
    dst_comp: B
```

Step 2: Parent SetServices Entry Point

File: `OuterMetaComponent/SetServices.F90` (Lines 29-100)

When parent gridcomp's SetServices is called, `SetServices_()` orchestrates the child creation:

```
recursive module subroutine SetServices_(this, rc)
   class(OuterMetaComponent), target, intent(inout) :: this
   integer, intent(out) :: rc

   ! STEP 1: Parse parent YAML into ComponentSpec
   this%component_spec = parse_component_spec(this%hconfig, this%registry, &
       this%user_gc_driver%get_name(), _RC)

   user_gridcomp = this%user_gc_driver%get_gridcomp()
   call attach_inner_meta(user_gridcomp, this%self_gridcomp, _RC)

   ! STEP 2: Call user's SetServices
   call this%user_setservices%run(user_gridcomp, _RC)

   ! STEP 3: Add children from parsed spec
   call add_children(this, _RC)

   ! STEP 4: Call SetServices on all children (recursive)
```

```
    call run_children_setservices(this, _RC)
 end subroutine SetServices_
```

Key nested subroutine for adding children:

```
 recursive subroutine add_children(this, rc)
    type(ChildSpecMapIterator) :: iter
    type(ChildSpec), pointer :: child_spec
    character(:), allocatable :: child_name

    associate ( e => this%component_spec%children%ftn_end() )
      iter = this%component_spec%children%ftn_begin()
      do while (iter /= e)
         call iter%next()
         child_name = iter%first()
         child_spec => iter%second()
         call this%add_child(child_name, child_spec, _RC)
      end do
    end associate
 end subroutine add_children
```

Step 3: Parent YAML Parsing - `parse_component_spec()`

File: `ComponentSpecParser/parse_component_spec.F90` (Lines 8-33)

Parses the parent's YAML into a `ComponentSpec` structure:

```
module function parse_component_spec(hconfig, registry, component_name, rc) &
    result(spec)
   type(ComponentSpec) :: spec
   type(ESMF_HConfig), target, intent(inout) :: hconfig
   character(*), intent(in) :: component_name
   integer, optional, intent(out) :: rc

   logical :: has_mapl_section
   type(ESMF_HConfig) :: mapl_cfg

   ! Navigate to the 'mapl:' section in parent YAML
   has_mapl_section = ESMF_HConfigIsDefined(hconfig, keyString=MAPL_SECTION, _RC)
   _RETURN_UNLESS(has_mapl_section)
   mapl_cfg = ESMF_HConfigCreateAt(hconfig, keyString=MAPL_SECTION, _RC)

   ! Parse all sections
   spec%geometry_spec = parse_geometry_spec(mapl_cfg, registry, component_name, _RC)
   spec%var_specs = parse_var_specs(mapl_cfg, registry, component_name, _RC)
   spec%connections = parse_connections(mapl_cfg, _RC)
   spec%children = parse_children(mapl_cfg, _RC)    ! <-- KEY: Parse children section
   spec%misc = parse_misc(mapl_cfg, _RC)

   call ESMF_HConfigDestroy(mapl_cfg, _RC)
end function parse_component_spec
```

Resulting ComponentSpec Data Structure (`specs/ChildSpec.F90`):

```
type :: ComponentSpec
  type(GeometrySpec) :: geometry_spec
  type(VariableSpecVector) :: var_specs
  type(ConnectionVector) :: connections
  type(ChildSpecMap) :: children    ! <-- Map: child_name => ChildSpec
  type(MiscellaneousComponentSpec) :: misc
end type ComponentSpec
```

Step 4: Children Section Iteration - `parse_children()`

File: `ComponentSpecParser/parse_children.F90` (Lines 9-45)

Iterates through the `children:` mapping in parent YAML:

```
module function parse_children(hconfig, rc) result(children)
   type(ChildSpecMap) :: children
   type(ESMF_HConfig), intent(in) :: hconfig
   integer, optional, intent(out) :: rc

   logical :: has_children, is_map
   type(ESMF_HConfig) :: children_cfg, child_cfg
   type(ESMF_HConfigIter) :: iter, iter_begin, iter_end
   type(ChildSpec) :: child_spec
   character(:), allocatable :: child_name
```

```
! Check if 'children:' section exists
has_children = ESMF_HConfigIsDefined(hconfig, keyString=COMPONENT_CHILDREN_SECTION, _RC)
_RETURN_UNLESS(has_children)   ! Return empty if no children

! Navigate to 'children:' section in parent YAML
children_cfg = ESMF_HConfigCreateAt(hconfig, keyString=COMPONENT_CHILDREN_SECTION, _RC)
is_map = ESMF_HConfigIsMap(children_cfg, _RC)
_ASSERT(is_map, 'children spec must be mapping')

! Iterate through each child (A, B, etc.)
iter_begin = ESMF_HConfigIterBegin(children_cfg, _RC)
iter_end = ESMF_HConfigIterEnd(children_cfg, _RC)
iter = iter_begin

do while (ESMF_HConfigIterLoop(iter, iter_begin, iter_end))
  ! Get child name (e.g., "A", "B")
  child_name = ESMF_HConfigAsStringMapKey(iter, _RC)

  ! Get child config section from parent YAML
  child_cfg = ESMF_HConfigCreateAtMapVal(iter, _RC)

  ! Parse individual child spec
  child_spec = parse_child(child_cfg, _RC)    ! <-- Parse from parent YAML

  ! Store in map
```

```
    call children%insert(child_name, child_spec)

    call ESMF_HConfigDestroy(child_cfg, _RC)
  end do

  call ESMF_HConfigDestroy(children_cfg, _RC)
end function parse_children
```

Step 5: Individual Child Parsing - `parse_child()`

File: `ComponentSpecParser/parse_child.F90` (Lines 8-70)

This is the **critical function** that reads child information from **parent YAML**:

```
module function parse_child(hconfig, rc) result(child)
  type(ChildSpec) :: child
  type(ESMF_HConfig), intent(in) :: hconfig    ! <-- Section from parent YAML
  integer, optional, intent(out) :: rc

  class(AbstractUserSetServices), allocatable :: setservices

  ! Define allowed key name variants
  character(*), parameter :: dso_keys(*) = [character(len=9) :: &
      'dso', 'DSO', 'sharedObj', 'sharedobj']
  character(*), parameter :: userProcedure_keys(*) = [character(len=10) :: &
      'SetServices', 'setServices', 'setservices']
```

```
integer :: i
character(:), allocatable :: dso_key, userProcedure_key, try_key
logical :: dso_found, userProcedure_found, has_key, has_config_file
type(ESMF_HConfig), allocatable :: child_hconfig
character(:), allocatable :: sharedObj, userProcedure, config_file
type(ESMF_TimeInterval), allocatable :: offset, timeStep

! ===== STEP 1: Read DSO name from parent YAML =====
dso_found = .false.
do i = 1, size(dso_keys)
   try_key = trim(dso_keys(i))
   has_key = ESMF_HconfigIsDefined(hconfig, keyString=try_key, _RC)
   if (has_key) then
      _ASSERT(.not. dso_found, 'multiple specifications for dso in hconfig for child')
      dso_found = .true.
      dso_key = try_key
   end if
end do
_ASSERT(dso_found, 'Must specify a dso for hconfig of child')
sharedObj = ESMF_HconfigAsString(hconfig, keyString=dso_key, _RC)
! Result: sharedObj = "libconfigurable_gridcomp"

! ===== STEP 2: Read SetServices routine name from parent YAML =====
userProcedure_found = .false.
do i = 1, size(userProcedure_keys)
```

```
   try_key = userProcedure_keys(i)
   if (ESMF_HconfigIsDefined(hconfig, keyString=try_key)) then
      _ASSERT(.not. userProcedure_found, 'multiple specifications for dso in hconfig for child')
      userProcedure_found = .true.
      userProcedure_key = try_key
   end if
end do
! Default to 'setservices_' if not specified in parent YAML
userProcedure = 'setservices_'
if (userProcedure_found) then
   userProcedure = ESMF_HconfigAsString(hconfig, keyString=userProcedure_key,_RC)
end if
! Result: userProcedure = "setservices_"

! ===== STEP 3: Read config_file path from parent YAML =====
has_config_file = ESMF_HconfigIsDefined(hconfig, keyString='config_file', _RC)
if (has_config_file) then
   config_file = ESMF_HconfigAsString(hconfig, keyString='config_file',_RC)
   ! Load child's config file (A.yaml) into HConfig
   child_hconfig = ESMF_HConfigCreate(filename=config_file,_RC)
   ! Result: child_hconfig contains A.yaml contents
end if

! ===== STEP 4: Create user_setservices object =====
setservices = user_setservices(sharedObj, userProcedure)
! Result: DSOSetServices object holding ("libconfigurable_gridcomp", "setservices_")
```

```
    ! ===== STEP 5: Parse optional timestep and offset =====
    call parse_timespec(hconfig, timeStep, offset, _RC)

    ! ===== STEP 6: Create ChildSpec =====
    child = ChildSpec(setservices, hconfig=child_hconfig, &
        timeStep=timeStep, offset=offset)

    _RETURN(_SUCCESS)
end function parse_child
```

nput (from parent YAML):

```
A:
  dso: libconfigurable_gridcomp
  config_file: scenarios/precision_extension/A.yaml
```

Output (ChildSpec for child A):

```
ChildSpec {
  user_setservices: DSOSetServices {
    sharedObj: "libconfigurable_gridcomp"
    userRoutine: "setservices_"
  }
  hconfig: <HConfig containing A.yaml content>
```

```
  timeStep: <not allocated>
  offset: 0 seconds
}
```

Step 6: User SetServices Factory - `user_setservices()`

File: `UserSetServices.F90` (Lines 135-149)

Creates a `DSOSetServices` object from the parsed parent YAML values:

```
! Argument names correspond to ESMF arguments.
function new_DSOSetServices(sharedObj, userRoutine) result(dso_setservices)
   use mapl_DSO_Utilities_mod
   type(DSOSetServices) :: dso_setservices
   character(len=*), intent(in) :: sharedObj
   character(len=*), optional, intent(in) :: userRoutine

   character(:), allocatable :: userRoutine_

   userRoutine_ = 'setservices_'  ! unless
   if (present(userRoutine)) userRoutine_ = userRoutine

   dso_setservices%sharedObj   = sharedObj
   dso_setservices%userRoutine = userRoutine_

end function new_DSOSetServices
```

Data Structure (`UserSetServices.F90` Lines 71-77):

```
type, extends(AbstractUserSetServices) :: DSOSetServices
   character(:), allocatable :: sharedObj    ! ESMF naming convention
   character(:), allocatable :: userRoutine  ! ESMF naming convention
contains
   procedure :: run => run_DSOSetServices
   procedure :: write_formatted => write_formatted_dso
end type DSOSetServices
```

Base Class (`UserSetServices.F90` Lines 30-35):

```
type, abstract :: AbstractUserSetServices
contains
   procedure(I_RunSetServices), deferred :: run
   procedure(I_write_formatted), deferred :: write_formatted
   generic :: write(formatted) => write_formatted
end type AbstractUserSetServices
```

Step 7: Child Addition to Parent - `add_child_by_spec()`

File: `OuterMetaComponent/add_child_by_spec.F90` (Lines 19-55)

Adds the child gridcomp to the parent after creation:

```
module recursive subroutine add_child_by_spec(this, child_name, child_spec, rc)
   class(OuterMetaComponent), target, intent(inout) :: this
   character(*), intent(in) :: child_name
   type(ChildSpec), intent(inout) :: child_spec
   integer, optional, intent(out) :: rc

   integer :: status
   type(GriddedComponentDriver) :: child_driver
   type(ESMF_GridComp) :: child_outer_gc
   type(OuterMetaComponent), pointer :: child_meta
   type(ESMF_HConfig) :: total_hconfig
   class(Logger), pointer :: lgr
   character(:), allocatable :: this_name

   _ASSERT(is_valid_name(child_name), 'Child name <' // child_name //'> does not conform to GEOS sta
   _ASSERT(this%children%count(child_name) == 0, 'duplicate child name: <'//child_name//'>.')

   ! Merge parent's HConfig with child's HConfig
   total_hconfig = merge_hconfig(this%hconfig, child_spec%hconfig, _RC)

   ! Create child gridcomp with:
   ! - child_name: "A"
   ! - child_spec%user_setservices: DSOSetServices("libconfigurable_gridcomp", "setservices_")
   ! - total_hconfig: Merged parent + child YAML
   child_outer_gc = MAPL_GridCompCreate(child_name, child_spec%user_setservices, &
       total_hconfig, _RC)
```

```
    ! Meta stuff
    child_meta => get_outer_meta(child_outer_gc, _RC)
    call this%registry%add_subregistry(child_meta%get_registry())

    if (allocated(child_spec%timeStep)) child_meta%user_timeStep = child_spec%timeStep

    child_meta%user_offset = this%user_offset + child_spec%offset

    child_driver = GriddedComponentDriver(child_outer_gc)
    call this%children%insert(child_name, child_driver)

    lgr => this%get_logger()
    this_name = this%get_name()
    call lgr%debug('%a added child <%a~>', this_name, child_name, _RC)

    _RETURN(_SUCCESS)
 end subroutine add_child_by_spec
```

Step 8: Child SetServices Execution (Recursive)

File: `OuterMetaComponent/SetServices.F90` (Lines 78-98)

After all children are added, their SetServices are called recursively:

```
recursive subroutine run_children_setservices(this, rc)
   class(OuterMetaComponent), target, intent(inout) :: this
   integer, optional, intent(out) :: rc

   integer :: status, user_status
   type(GriddedComponentDriver), pointer :: child_comp
   type(ESMF_GridComp) :: child_outer_gc
   type(GriddedComponentDriverMapIterator) :: iter

   associate ( e => this%children%ftn_end() )
      iter = this%children%ftn_begin()
      do while (iter /= e)
         call iter%next()
         child_comp => iter%second()
         child_outer_gc = child_comp%get_gridcomp()
         ! Call SetServices on child (recursively calls SetServices_ again)
         call ESMF_GridCompSetServices(child_outer_gc, mapl_GenericSetServices, _USERRC)
      end do
   end associate

   _RETURN(ESMF_SUCCESS)
end subroutine run_children_setservices
```

When `run_DSOSetServices()` is called on each child's `DSOSetServices` object:

```
subroutine run_DSOSetServices(this, gridcomp, rc)
   use mapl_DSO_Utilities_mod
   class(DSOSetservices), intent(in) :: this
   type(ESMF_GridComp) :: GridComp
   integer, intent(out) :: rc

   integer :: status, user_status
   logical :: found

   _ASSERT(is_supported_dso_name(this%sharedObj), 'unsupported dso name:: <'//this%sharedObj//'>')
   ! Dynamically load .so and call user's SetServices routine
   call ESMF_GridCompSetServices(gridcomp, sharedObj=adjust_dso_name(this%sharedObj), &
        userRoutine=this%userRoutine, userRoutinefound=found, _USERRC)

   _RETURN(ESMF_SUCCESS)
end subroutine run_DSOSetServices
```

Data Structures

ChildSpec

File: `specs/ChildSpec.F90` (Lines 16-24)

```
type :: ChildSpec
   class(AbstractUserSetServices), allocatable :: user_setservices
```

```
   type(ESMF_HConfig) :: hconfig                  ! Child's own YAML config
   type(ESMF_TimeInterval), allocatable :: timeStep
   type(ESMF_TimeInterval) :: offset
contains
   procedure :: write_formatted
   generic :: write(formatted) => write_formatted
end type ChildSpec
```

Constructor:

```
function new_ChildSpec(user_setservices, unusable, hconfig, timeStep, offset) &
    result(spec)
   type(ChildSpec) :: spec
   class(AbstractUserSetServices), intent(in) :: user_setservices
   class(KeywordEnforcer), optional, intent(in) :: unusable
   type(ESMF_HConfig), optional, intent(in) :: hconfig
   type(ESMF_TimeInterval), optional, intent(in) :: timeStep
   type(ESMF_TimeInterval), optional, intent(in) :: offset

   spec%user_setservices = user_setservices
   if (present(hconfig)) then
      spec%hconfig = hconfig
   else
      spec%hconfig = ESMF_HConfigCreate(content='{}')
   end if
```

```
   call ESMF_TimeIntervalSet(spec%offset, s=0)
   if (present(timeStep)) spec%timeStep = timeStep
   if (present(offset)) spec%offset = offset
end function new_ChildSpec
```

DSOSetServices

File: `UserSetServices.F90` (Lines 71-77)

```
type, extends(AbstractUserSetServices) :: DSOSetServices
   character(:), allocatable :: sharedObj    ! e.g., "libconfigurable_gridcomp"
   character(:), allocatable :: userRoutine  ! e.g., "setservices_"
contains
   procedure :: run => run_DSOSetServices
   procedure :: write_formatted => write_formatted_dso
end type DSOSetServices
```

AbstractUserSetServices (Base Class)

File: `UserSetServices.F90` (Lines 30-35)

```
type, abstract :: AbstractUserSetServices
contains
   procedure(I_RunSetServices), deferred :: run
   procedure(I_write_formatted), deferred :: write_formatted
   generic :: write(formatted) => write_formatted
```

```
end type AbstractUserSetServices
```

nterface:

```
abstract interface
   subroutine I_RunSetServices(this, gridcomp, rc)
      use esmf, only: ESMF_GridComp
      import AbstractUserSetServices
      class(AbstractUserSetServices), intent(in) :: this
      type(ESMF_GridComp) :: gridcomp
      integer, intent(out) :: rc
   end subroutine I_RunSetServices
end interface
```

Complete Execution Timeline

Example: Creating Child "A"

Step	Location	Action	Input	Output
1	`SetServices_()`	Parse parent YAML	parent.yaml in HConfig	ComponentSpec with children map
2	`parse_children()`	Iterate children section	`children:` mapping	ChildSpec for each child
3	`parse_child()`	Read child "A" spec from parent YAML	`A: {dso: libconfigurable_gridcomp, config_file: A.yaml}`	Read dso, SetServices, config_file paths
4	`parse_child()`	Load child's YAML file	config_file path: `A.yaml`	child_hconfig with A.yaml contents

5	`user_setservices()`	Create DSOSetServices	sharedObj, userRoutine	`DSOSetServices("libconfigurable_gridcomp", "setservices_")`
6	`parse_child()`	Create ChildSpec	DSOSetServices + child_hconfig	ChildSpec for child A
7	`add_child_by_spec()`	Create child gridcomp	ChildSpec, child name	childoutergc (ESMF gridcomp)
8	`add_child_by_spec()`	Register child	childoutergc	Added to parent's children map
9	`run_children_setservices()`	Call child's SetServices	child gridcomp	`run_DSOSetServices()` called
10	`run_DSOSetServices()`	Load DSO and call SetServices	sharedObj, userRoutine	DSO loaded, user's `setservices_` routine called
11	(Recursive)	Child's SetServices	child gridcomp, child_hconfig	Same process repeats for child's children

---

Key Files Summary

Component	File	Function/Subroutine	Lines	Purpose
Parent Creation	`GenericGridComp.F90`	`create_grid_comp_primary()`	87-142	Create parent gridcomp, store YAML in OuterMetaComponent
Parent SetServices Orchestration	`OuterMetaComponent/SetServices.F90`	`SetServices_()`	29-100	Orchestrate parsing and child creation
Parse Component Spec	`ComponentSpecParser/parse_component_spec.F90`	`parse_component_spec()`	8-33	Parse entire parent YAML into ComponentSpec
Parse Children Section	`ComponentSpecParser/parse_children.F90`	`parse_children()`	9-45	Iterate children mapping in parent YAML
Parse Individual Child	**`ComponentSpecParser/parse_child.F90`**	**`parse_child()`**	**8-70**	**Read dso, SetServices, config_file from parent YAML**
Create SetServices Factory	`UserSetServices.F90`	`new_DSOSetServices()`	135-149	Create DSOSetServices object
Add Child to Parent	`OuterMetaComponent/add_child_by_spec.F90`	`add_child_by_spec()`	19-55	Create child gridcomp and register with parent

Child SetServices Execution	`OuterMetaComponent/SetServices.F90`	`run_children_setservices()`	78-98	Call SetServices recursively on children
DSO SetServices Execution	`UserSetServices.F90`	`run_DSOSetServices()`	151-165	Load DSO and call user's SetServices routine
Child Spec Definition	`specs/ChildSpec.F90`	ChildSpec type	16-24	Data structure holding child specification
DSO SetServices Type	`UserSetServices.F90`	DSOSetServices type	71-77	Polymorphic type holding .so name and routine name
Abstract Base Class	`UserSetServices.F90`	AbstractUserSetServices type	30-35	Abstract base for SetServices variants

---

Key Insights

1. **Parent Drives Child Creation**: Parent's YAML contains all child specifications in the `children:` section
2. **Three Pieces of Information**: For each child, parent YAML specifies:
   - `dso` (or `DSO`, `sharedObj`, `sharedobj`): Shared object library name
   - `SetServices` (optional, defaults to `setservices_`): Name of SetServices routine in the DSO
   - `config_file`: Path to child's own configuration YAML file
3. **Hierarchical YAML Loading**:
   - Parent YAML is loaded into HConfig
   - For each child, child's YAML (from `config_file`) is loaded separately
   - Parent's and child's HConfigs are merged in `add_child_by_spec()`
4. **Polymorphic SetServices**: `AbstractUserSetServices` allows different SetServices implementations (currently `ProcSetServices` and `DSOSetServices`)
5. **Recursive Process**: SetServices is called recursively on each level, allowing arbitrary depth of nesting
6. **DSO Loading Deferred**: DSOs are not loaded until `run_DSOSetServices()` is called, allowing for runtime specification

---

Example: Complete YAML Configuration

Parent YAML

```yaml
grid:
  class: LatLon
  im_world: 12
  jm_world: 6

children:
  PhysicsChild:
    dso: libphysics_gridcomp
    SetServices: my_custom_setservices  # Optional, defaults to setservices_
    config_file: physics/config.yaml

  DiagnosticsChild:
    dso: libdiagnostics_gridcomp
    config_file: diagnostics/config.yaml

states: {}

connections:
  - src_name: T
    src_comp: PhysicsChild
    dst_name: TEMP
    dst_comp: DiagnosticsChild
```

Child YAML (physics/config.yaml)

```yaml
mapl:
  states:
    - Field1:
        long_name: "Temperature"
        units: K
        dims: [xy, z]

  children:
    SubModel:
      dso: libsubmodel_gridcomp
      config_file: submodels/config.yaml
```

Execution Flow

1. Parent SetServices reads parent.yaml
2. Finds `PhysicsChild` and `DiagnosticsChild` in `children:`
3. For `PhysicsChild`:
   - Reads `dso: libphysics_gridcomp`
   - Reads `SetServices: my_custom_setservices`
   - Reads `config_file: physics/config.yaml`
   - Loads physics/config.yaml
   - Creates `DSOSetServices("libphysics_gridcomp", "my_custom_setservices")`
   - Creates child gridcomp
4. For `DiagnosticsChild`:
   - Reads `dso: libdiagnostics_gridcomp`
   - Defaults to `SetServices: setservices_`
   - Reads `config_file: diagnostics/config.yaml`
   - Loads diagnostics/config.yaml
   - Creates `DSOSetServices("libdiagnostics_gridcomp", "setservices_")`

    - Creates child gridcomp
5. Calls SetServices on both children recursively
6. `PhysicsChild`'s SetServices executes, finds its own `SubModel` child, repeats process